

Problem 1

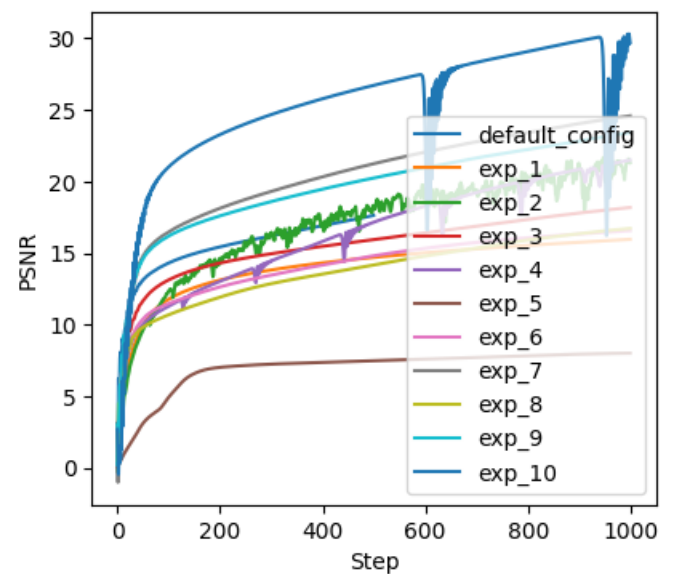
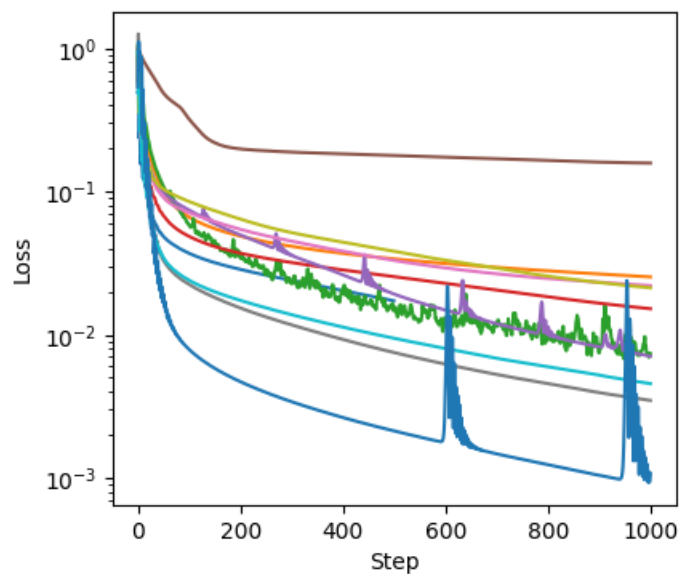
Short Answer

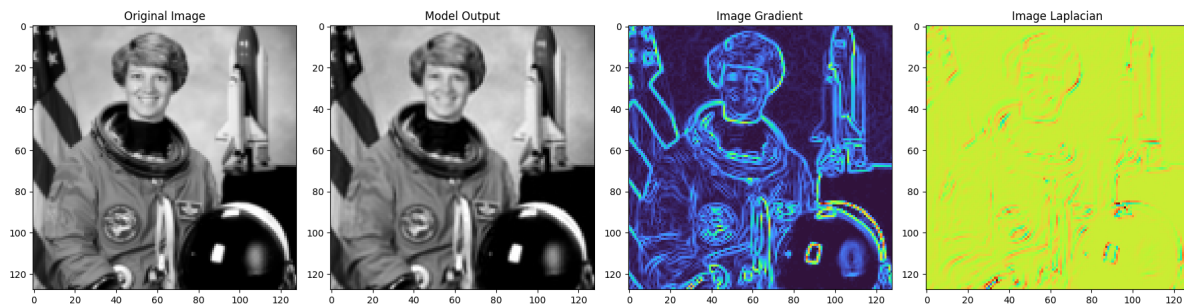
MLP reconstructions are much less detailed than those produced by SIREN due to differences in activation functions and their ability to model high-frequency variations. MLPs typically use activations like ReLU, Tanh, or Sigmoid, which struggle to represent fine details; ReLU in particular produces piecewise linear approximations, while Tanh and Sigmoid suffer from vanishing gradients, making it difficult to capture high-frequency structures. In contrast, SIREN uses sinusoidal activations, which naturally encode high-frequency information and allow the network to represent fine textures and edges more effectively. The sine function's Fourier properties enable SIREN to model gradients efficiently as well, resulting in more detailed and accurate reconstructions.

The Laplacian of an MLP-generated image appears strange because standard MLPs are biased toward low frequencies and struggle with second-order derivatives. MLPs learn smooth, low-frequency components first, leading to an unnatural Laplacian that lacks sharp details. Additionally, ReLU-based MLPs produce piecewise-linear outputs, whose second derivatives are mostly zero except at discontinuities, creating patchy and unnatural Laplacian representations. In contrast, SIREN lends itself naturally to further-order derivatives via sine activation, yielding a Laplacian that more accurately captures smooth transitions and oscillatory variations. This fundamental difference in how MLPs and SIRENs model image structure explains the poor Laplacian quality in MLPs and the superior reconstruction capabilities of SIREN.

Experiments

Exp. #	Changes	Notes
1	lower LR ($lr=1e-5$)	likely more stable, just needs more training steps
2	higher LR ($lr=1e-3$)	converges faster for this one example, but less stable loss curve
3	fewer hidden features (hidden_features=64)	doesn't converge quickly, output image and gradient both blurry/not expressive
4	more hidden features (hidden_features=256)	produces better output image, but there seem to be some artifacts/shortcuts in gradient image
5	1 hidden layer (less)	completely unintelligible, blurry/bloppy output
6	2 hidden layers (less)	pixeled/blurry output with significant artifacts in output image, but image starting to form
7	5 hidden layers (more)	excellent output, gradient closer resembles the output as well (i.e. sharper lines)
8	lower initial/hidden omega_0 (omega_0=10.0)	blurry output, similar to hidden_layers=2
9	higher initial/hidden omega_0 (omega_0=30.0)	better output than default config, sharper gradient image, but has some artifacts in output and gradient
10 (best)	default LR, more hidden features, more hidden layers, higher omega_0	excellent output, gradient and laplacian images both have sharper edges than previous experiments, though curves do spike; in general, it seems that increasing the general size of the NN improves performance, as expected, and a higher omega_0 of 30.0 produces a better output for this image in particular





```
exp10_loss, exp10_psnr = train(  
    model="SIREN",  
    dataset=dataset,  
    lr=1e-4,  
    total_steps=1000,  
    steps_til_summary=100,  
    device=device,  
    **dict(  
        in_features=2,  
        out_features=1,  
        hidden_features=256,  
        hidden_layers=5,  
        first_omega_0=30.0,  
        hidden_omega_0=30.0  
    ),  
)
```